

Exercise 1: Naive Bayes Classifier from Scratch

You are required to implement a **Gaussian Naive Bayes classifier from scratch** (without using `scikit-learn` or any pre-built library). The dataset contains 30 rows, where each row has 8 feature values, followed by a binary class label (0 or 1), and an extra column to be ignored.

Requirements

1. Read n rows of input data using `input().split()`.
2. Split the dataset into training (70%) and testing (30%) sets using the first 70% rows for training and the remaining 30% rows for testing.
3. Compute the following from the training data:
 - Class priors
 - Mean and standard deviation of each feature per class (used internally, not printed)
4. For each test sample:
 - Use the Gaussian probability density function for each feature.
 - Multiply the probabilities with the class prior to compute class likelihoods.
 - Assign the class with maximum likelihood.
5. Print the following results (rounded to 2 decimal places):
 - Number of training and test samples
 - Class priors
 - Predictions vs Actual labels for the test set
 - Final evaluation metrics: Accuracy, Precision, Recall, and F1-score

Dataset (30 rows)

Each row contains:

Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age, Outcome, Outlier

```
4 122 61 7 52 27.82219568 0.635762173 48 0 0
1 104 56 11 0 24.24503026 0.089848385 60 0 0
3 109 62 23 0 18.87690204 0.332317171 61 0 0
3 117 87 21 108 30.4921571 0.405407555 25 1 0
2 114 56 28 10 23.1479042 0.120255598 25 0 0
3 120 95 13 165 32.39493617 0.211376261 26 1 0
2 99 59 20 3 44.38536126 0.219177313 43 0 0
3 150 78 13 116 33.39655129 0.309157203 23 0 0
0 122 68 27 19 45.98096352 0.298969424 57 0 0
2 104 66 31 110 25.12408723 0.494678442 53 0 0
4 130 54 24 0 23.0334144 0.340692543 63 1 0
2 117 68 30 43 24.81089088 0.260540091 41 0 0
1 67 56 20 12 31.54429623 0.116002259 40 0 0
3 130 49 28 150 51.06064081 0.081957536 21 0 0
4 89 71 20 116 20.36519701 0.105820288 59 1 0
4 148 62 9 0 36.33764183 0.559926645 67 1 0
1 112 90 22 63 27.10024265 0.679502021 37 1 0
2 143 82 10 18 30.43337864 0.145064816 49 0 0
3 101 90 43 156 23.19386062 0.165994 28 1 0
2 122 61 16 181 38.15396827 0.491571652 46 1 0
3 156 71 26 0 31.69252407 0.039152253 48 0 0
4 111 38 19 146 39.61217485 0.278828256 54 0 0
1 102 66 7 14 33.22671636 0.462085911 46 0 0
3 139 71 7 59 18.47863216 0.090168205 33 0 0
3 139 71 27 82 39.67579957 0.238599802 64 0 0
3 103 69 35 225 17.90899557 0.501099654 57 0 0
5 60 70 12 0 18.91068122 0.155102847 33 1 0
0 101 37 7 146 25.57958124 0.096417867 58 0 0
1 140 73 16 0 25.67301121 0.255078432 45 1 0
4 96 75 10 0 41.77214956 0.376397231 67 0 0
```

Expected Output Format

Training samples: XX

Test samples: XX

Class 0 Prior: 0.62

Class 1 Prior: 0.38

Predictions: [0, 1, 0, 0, 1, ...]

Actual: [0, 0, 0, 1, 1, ...]

Accuracy=0.56

Precision=0.00

Recall=0.00

F1=0.00

Exercise 2: Multinomial Naive Bayes Classifier from Scratch

You are required to implement a Multinomial Naive Bayes classifier from scratch (without using scikit-learn or any pre-built library). The dataset contains 24 rows, where each row is a short text document labeled as Positive (1) or Negative (0).

Requirements

1. Read n rows of input data using `input().split()`.
2. Split the dataset into training (70%) and testing (30%) sets using the first 70% rows for training and the remaining 30% rows for testing.
3. Compute the following from the training data:

- Class priors:

$$P(C) = \frac{\text{Number of documents in class } C}{\text{Total documents}}$$

- Vocabulary of all unique words.
- Word frequencies for each class.
- Conditional probabilities with Laplace smoothing:

$$P(w \mid C) = \frac{\text{Count}(w, C) + 1}{\sum_{w'} \text{Count}(w', C) + |V|}$$

where $|V|$ is the vocabulary size.

4. For each test sample:
 - Use the Multinomial Naive Bayes formula:

$$P(C \mid d) \propto P(C) \cdot \prod_{w \in d} P(w \mid C)^{\text{count}(w, d)}$$

- Assign the class with maximum likelihood.
5. Print the following results (rounded to 2 decimal places):

- Number of training and test samples
- Class priors
- Predictions vs Actual labels for the test set
- Final evaluation metrics: Accuracy, Precision, Recall, and F1-score

Dataset (24 rows)

Each row contains: Text , Label

```
I love this movie , 1
This film was great , 1
Amazing acting and story , 1
What a fantastic performance , 1
The direction was brilliant , 1
Highly enjoyable and emotional , 1
I really liked the soundtrack , 1
The characters were inspiring , 1
This was a boring movie , 0
I did not like the film , 0
Terrible acting and weak story , 0
The performance was disappointing , 0
It was a waste of time , 0
Not enjoyable at all , 0
I hated the soundtrack , 0
The characters were dull , 0
Amazing visuals but boring story , 0
Loved the emotional moments , 1
Brilliant soundtrack and acting , 1
Terrible direction and slow pace , 0
Enjoyable family film , 1
Weak characters and dull ending , 0
Fantastic film overall , 1
Not worth watching , 0
```

Expected Output Format

Training samples: XX

Test samples: XX
Class 0 Prior: 0.50
Class 1 Prior: 0.50
Predictions: [1, 0, 1, ...]
Actual: [1, 0, 0, ...]
Accuracy=0.67
Precision=0.60
Recall=0.75
F1=0.67